Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ-1 «Системы автоматического управления»

ОТЧЕТ

по лабораторной работе №4

«Поиск объекта методами спектральной корреляции»

по дисциплине

«Распознавание образов»

Вариант 4

Выполнили: Хань Цзиньчжоу

Группа: ИУ1И-31М

Проверил: Водопьянов В.С.

Работа выполнена: 26/10/2026

Отчет сдан: 26/10/2026

Оценка:

Москва 2026

СОДЕРЖАНИЕ

1. Теоретические сведения 3

1.1 Нормализация яркости 3

1.2 Фазовая корреляция 4

1.3 Градиентная корреляция 4

1.4 Контурная корреляция 5

2. Задания лабораторной работы 6

2.1 Прямое вычисление корреляционной функции 6

2.2 Спектральный анализ корреляции 6

2.3 Фазовая корреляция 7

2.4 Градиентная корреляция 12

2.5 Контурная корреляция 16

2.6 Сравнение 19

3. Код 20

1. Теоретические сведения

Как сравнить полученный снимок с реальной картой и определить место съёмки? Наиболее очевидное решение — попиксельно сравнить снимок с различными участками карты, а затем подсчитать число совпадающих точек. Однако снимок и карта получены в разное время и при разных условиях освещения, поэтому получить значительное число совпадающих точек затруднительно.

Другой способ заключается в использовании двумерной функции взаимной корреляции:

$$R(x,y)=\Sigma\Sigma\, A(x+dx,y+dy)*B(dx,dy)$$

Для монохромных изображений функция взаимной корреляции показывает число совпадающих белых пикселей при заданном положении. Недостатком является очень большой объём вычислений.

Для оптимизации можно использовать теорему о свёртке: заменить операцию свёртки во временной области умножением в частотной области и тем самым уменьшить объём вычислений.

$$F(A\otimes B)=F(A)xF^*(B),$$

Таким образом, формулу вычисления функции взаимной корреляции можно представить как обратное преобразование Фурье результата умножения в частотной области.

$$R(A,B)=F^{-1}[F(A)xF^*(B)],$$

Кроме того, условия освещения существенно влияют на яркость снимка и карты. Даже небольшие различия освещения могут приводить к очень большим ошибкам, поэтому необходимо снизить чувствительность вычислений к условиям освещения.

1.1 Нормализация яркости

Простейший способ — вычислить среднюю яркость всего изображения и вычесть её из яркости каждого пикселя. Однако распределение яркости

реального изображения не является гауссовским: различные участки местности имеют разную среднюю яркость. Поэтому для нормализации каждой точки необходимо вычислять среднюю яркость в её окрестности. Сложность вычисления средней яркости можно снизить алгоритмически.

1.2 Фазовая корреляция

Из теоремы о сдвиге следует, что при чистом смещении на dt, без поворота и изменения масштаба, амплитуда спектра не меняется, а изменяется только фаза. Следовательно, различие между положением изображения и положением соответствующего участка на карте отражается только в фазе. Амплитуда же содержит информацию о шуме, освещении и других факторах.

$$F[X(t+dt)] = F[X(t)]\ \exp(*-2\pi j\ dt)$$

Поэтому для получения функции взаимной корреляции можно выделить фазу взаимного спектра.

$$R(A,B)=F^{-1}[F(A)xF^*(B) / |F(A)xF^*(B)|]$$

Если изображение полностью совпадает с некоторым участком карты, функция взаимной корреляции имеет вид дельта-функции: на нулевом фоне присутствует единственный пик. При этом амплитуда не учитывается, поэтому метод нечувствителен к условиям освещения и другим шумам.

1.3 Градиентная корреляция

При изменении освещения градиент яркости практически не меняется. Поэтому для устранения влияния освещения на результаты анализа можно использовать корреляцию между градиентами яркости наряду с взаимной корреляцией самой яркости.

$$R1(A,B)=F^{-1}[F(G_A)xF^*(G_B)],$$

$$R2(A,B)=F^{-1}[F(G_{Ax})xF^*(G_{Bx}) + (G_{Ay})xF^*(G_{By})],$$

где $G_{Ax}=dA/dx$, $G_{Ay}=dA/dy$, $G_{Bx}=dB/dy$ $G_{By}=dB/dy$, – производные карты A и кадра B по x и по y соответственно,

$G_A=\sqrt{(G_{Ax}^2+G_{Ay}^2)}$ – модуль градиента карты A,

$G_B=\sqrt{(G_{Bx}^2+G_{By}^2)}$ – модуль градиента кадра B.

Модуль градиента можно использовать для вычисления корреляции, а его направление — для исследования угла поворота снимка относительно карты.

Градиентная корреляция, как и фазовая, нечувствительна к шуму и изменениям освещения и имеет низкую чувствительность к геометрическим искажениям.

1.4 Контурная корреляция

Контурная корреляция использует для распознавания только геометрические признаки, поэтому принципиально не зависит от изменений освещения. Корреляционную кривую можно получить с помощью алгоритма Canny. Чувствительность контурной корреляции к геометрическим искажениям ниже, чем у фазовой корреляции, однако она чувствительна к шуму.

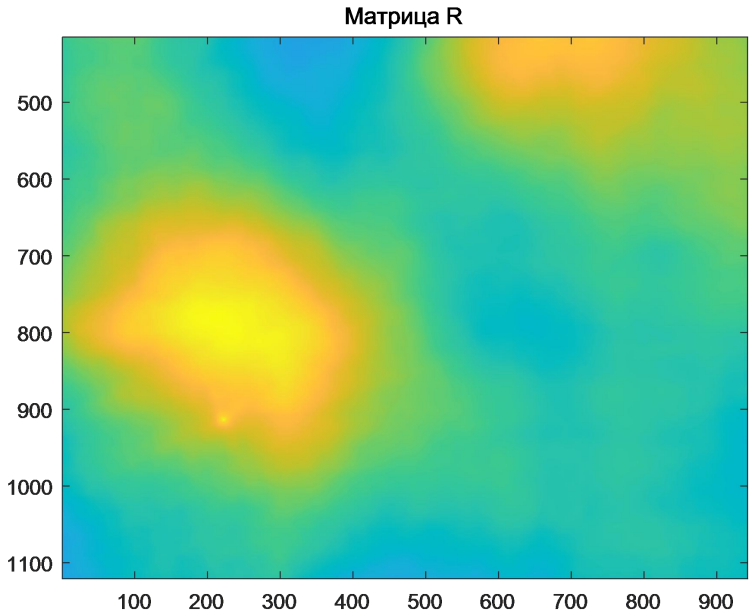
2. Задания лабораторной работы

2.1 Прямое вычисление корреляционной функции

Вычисление взаимной корреляции выполняется успешно, но очень медленно: полный расчёт на моём компьютере занял 1001 с.

2.2 Спектральный анализ корреляции

Полный расчёт этим методом занял 0,255 с, что в 3927 раз быстрее прямого вычисления. Это практически позволяет работать в реальном времени.



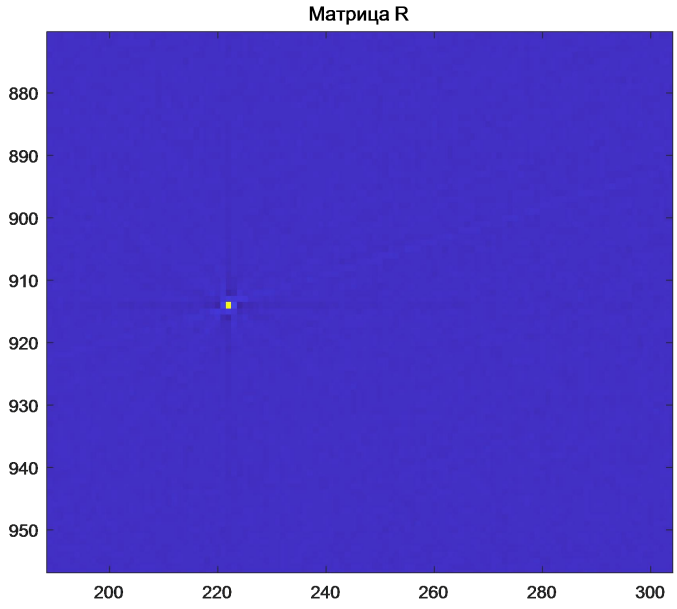


Однако результат неверен, возможно, из-за влияния условий освещения. Далее выполняется сглаживание освещения.



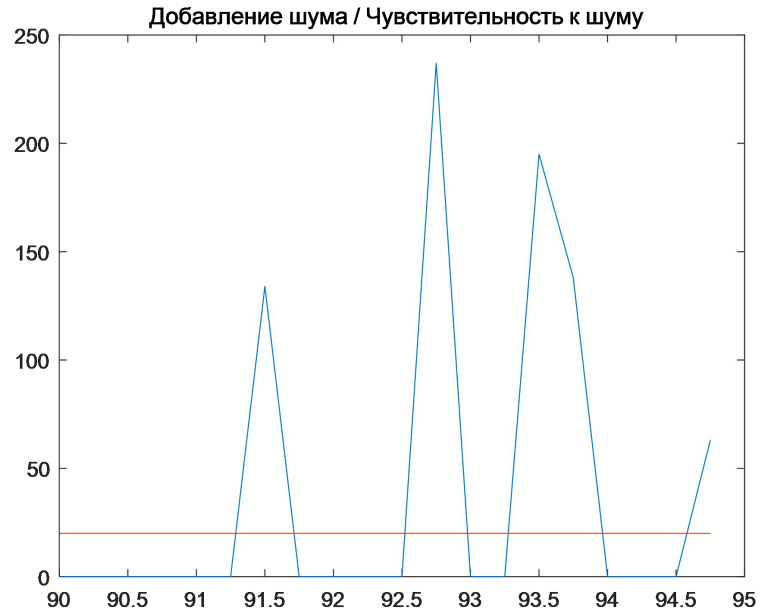
Теперь положение определяется правильно.

2.3 Фазовая корреляция

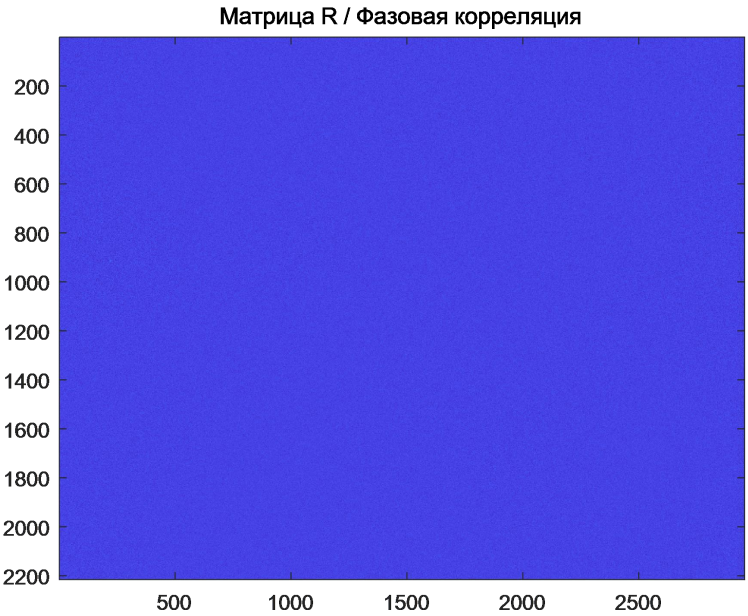


В этом случае корреляционная функция содержит только одну яркую точку на полностью нулевом фоне, причём изменение условий освещения не влияет на корреляцию.

Результат при добавлении шума показан на рисунке. При отдельных значениях возникают ошибки.

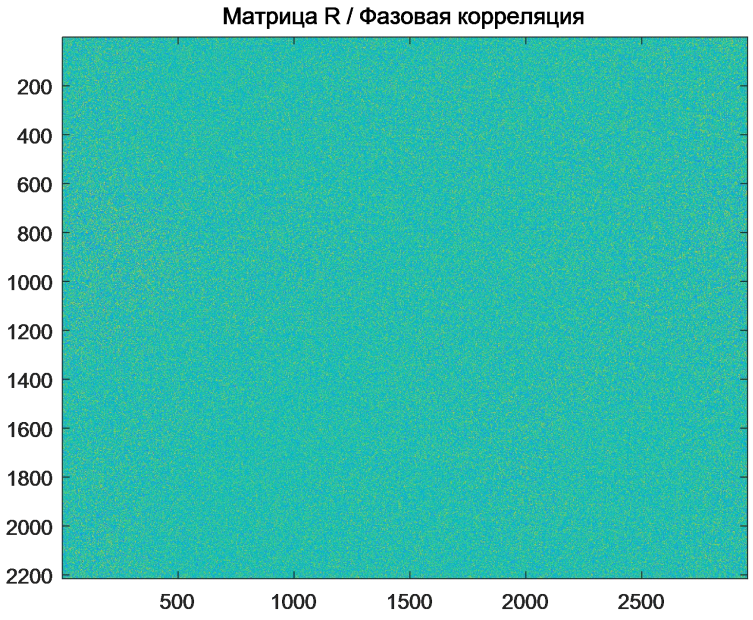


При добавлении шума интенсивностью 0,5 результат обнаружения остаётся правильным.



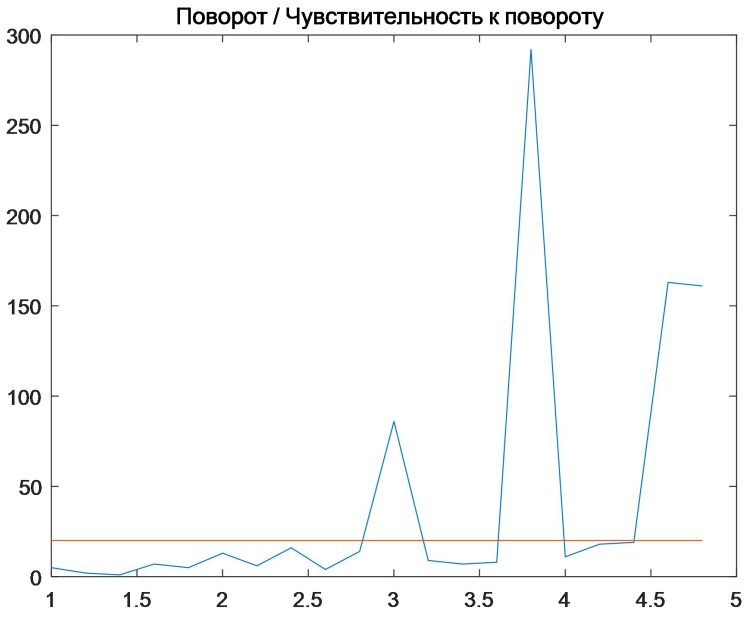


При интенсивности шума 0,97 правильное обнаружение невозможно.

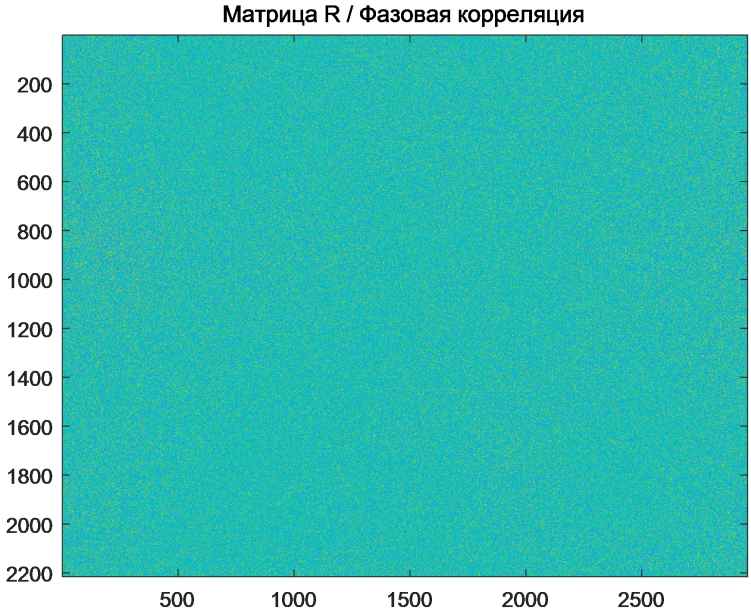




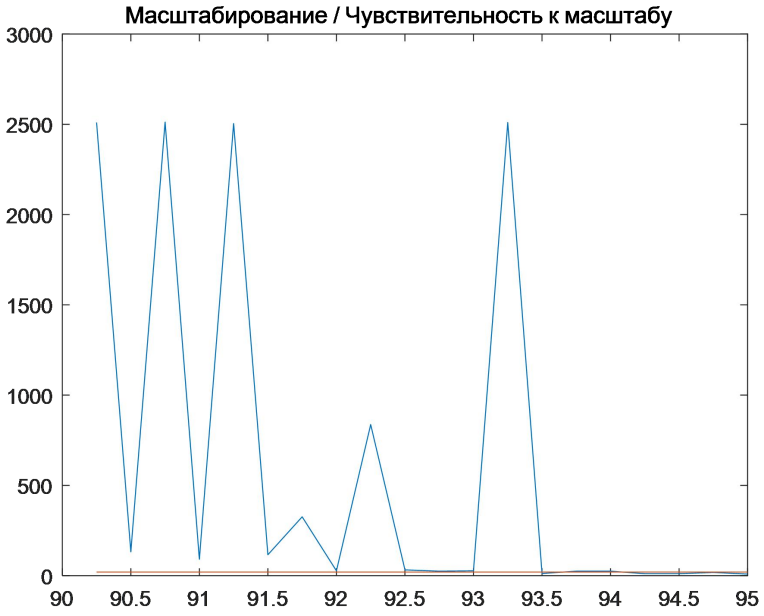
Результат после добавления поворота показан на рисунке.



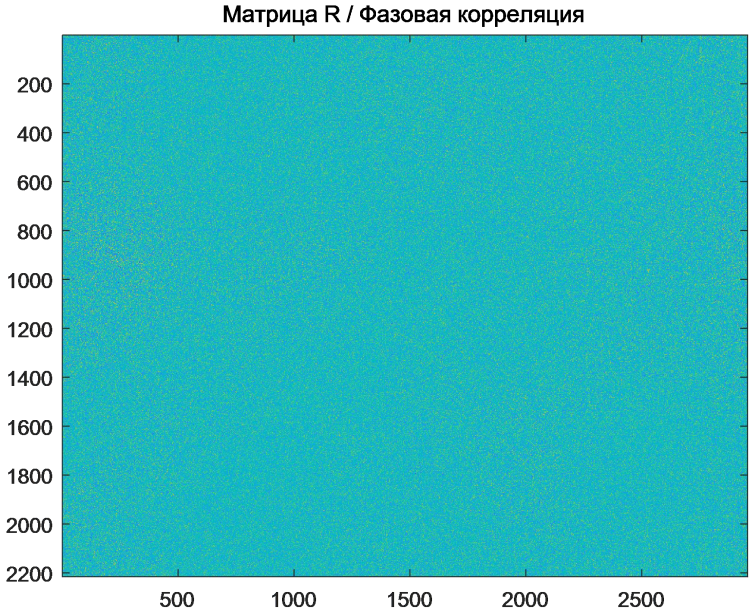
После поворота на 5° против часовой стрелки корреляционная функция имеет показанный на рисунке вид, без выраженного пика.



Результат после изменения масштаба показан на рисунке.

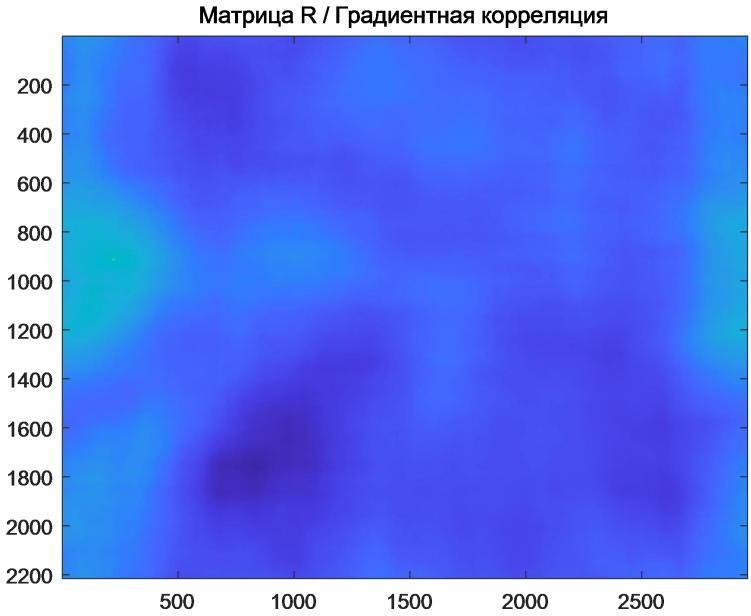


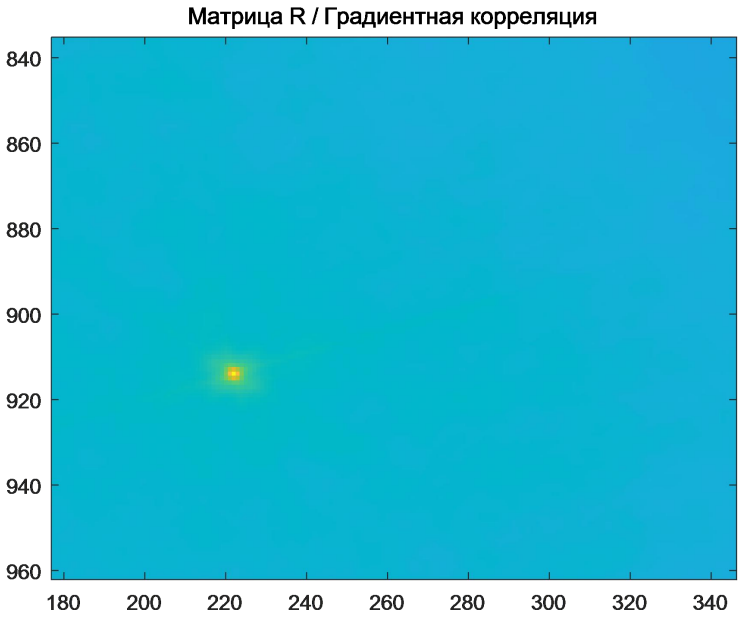
При уменьшении масштаба до 0,9 корреляционная функция имеет показанный на рисунке вид, без выраженного пика.

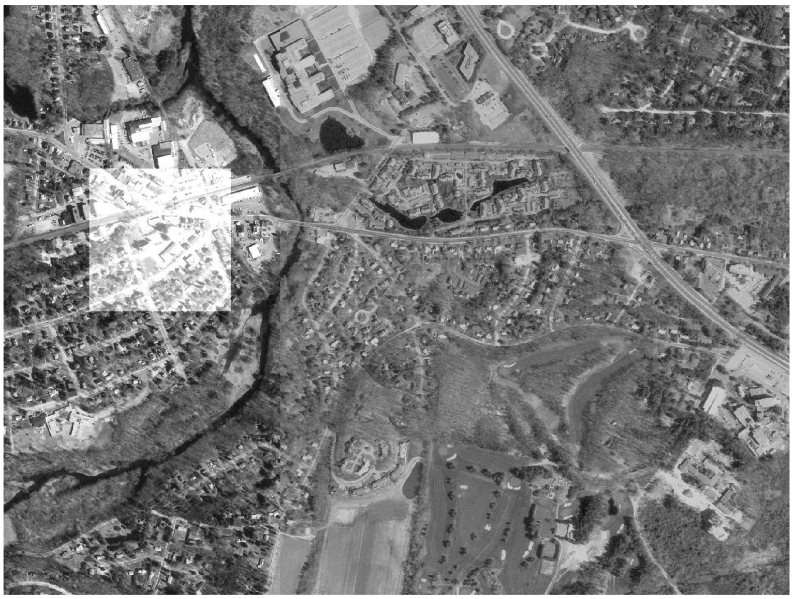


2.4 Градиентная корреляция

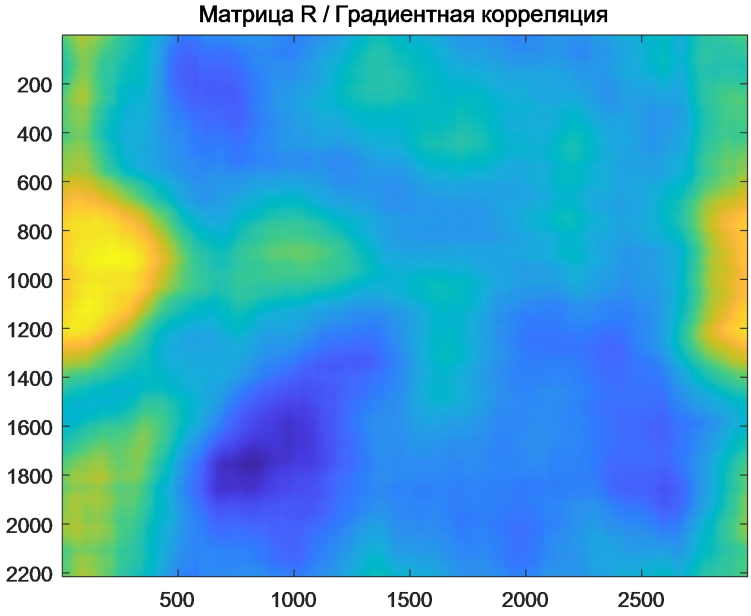
Корреляционная функция также содержит выраженную яркую точку, и результат распознавания является правильным.

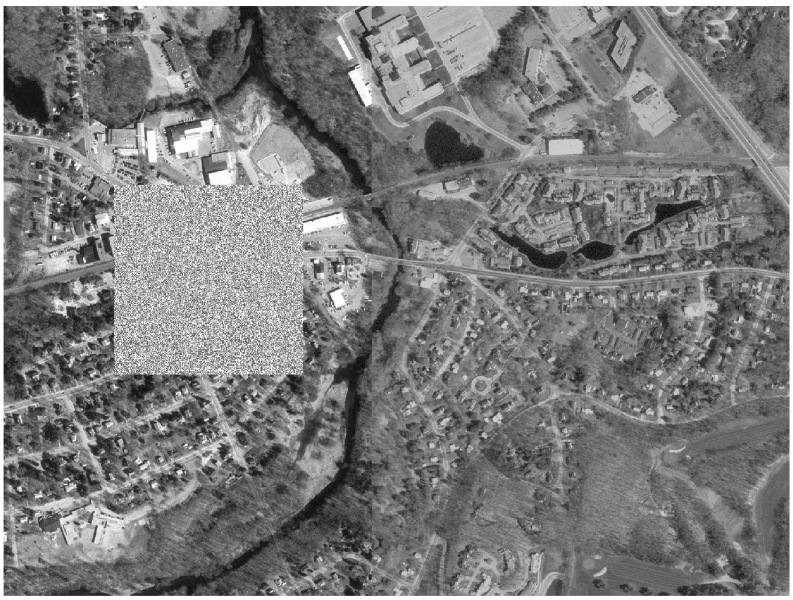



Матрица R / Градиентная корреляция
840
860
880
900
920
940
960
180
200
220
240
260
280
300
320
340

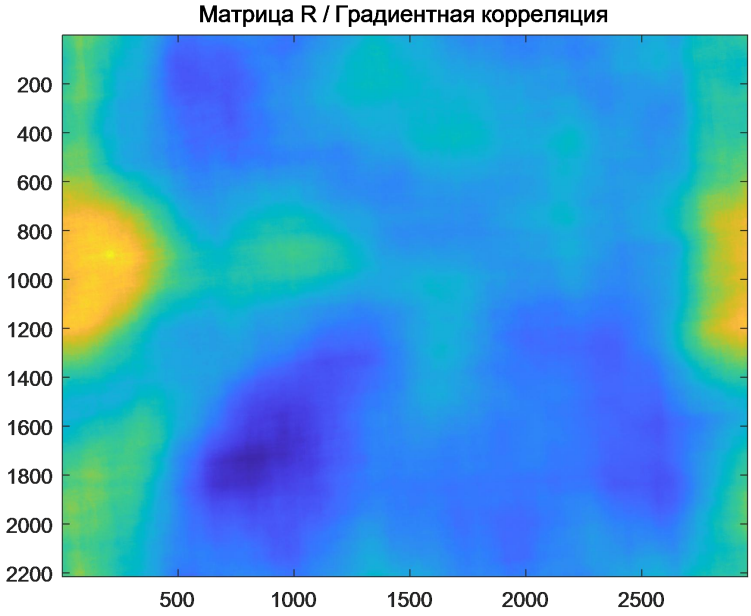


При добавлении шума интенсивностью 0,97 по-прежнему удаётся получить правильный результат.

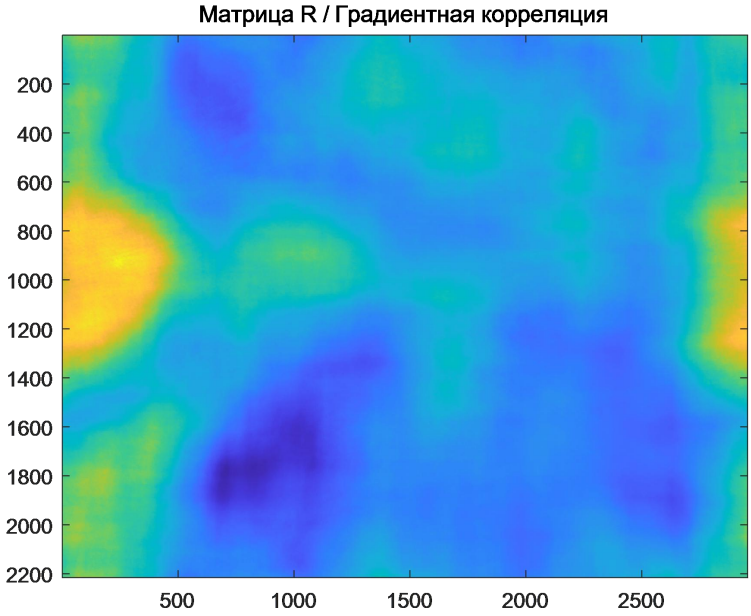




После поворота на 5° против часовой стрелки корреляционная функция по-прежнему позволяет определить правильное положение.

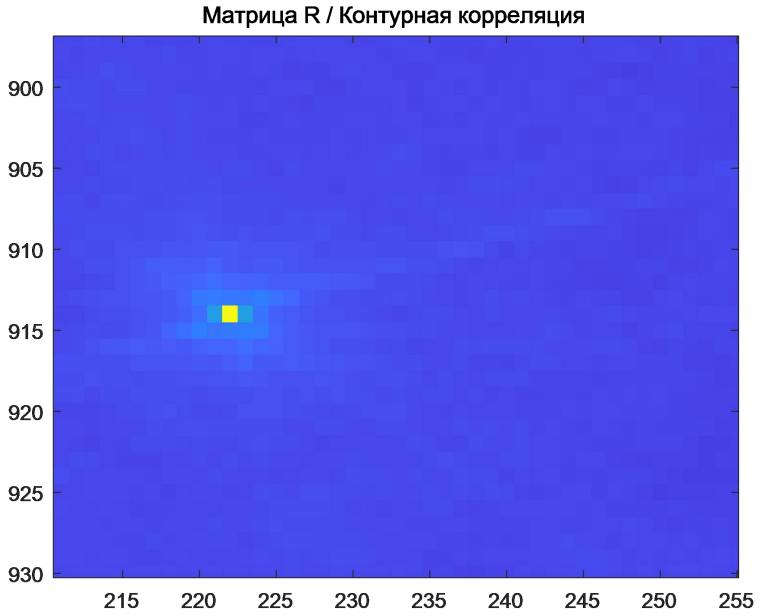


При уменьшении масштаба до 0,9 по-прежнему удаётся получить достаточно точный результат.



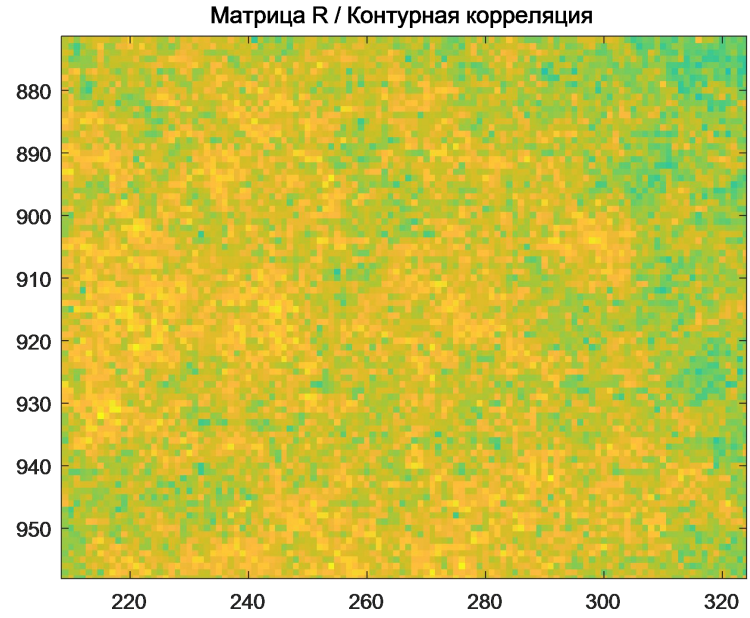
2.5 Контурная корреляция

Корреляционная функция достигает пика в очень точно определённой точке.

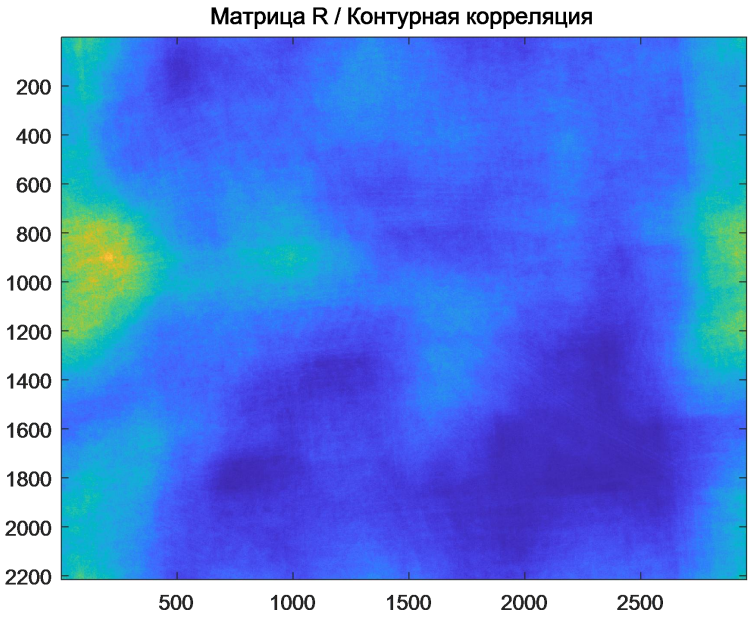




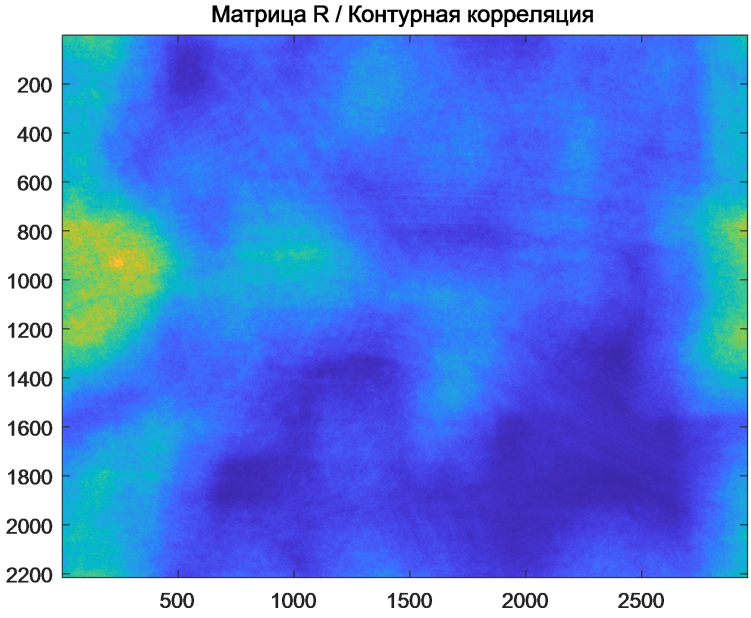
Однако метод очень чувствителен к шуму: после добавления шума интенсивностью 0,5 получить точный результат корреляционной функции практически невозможно.



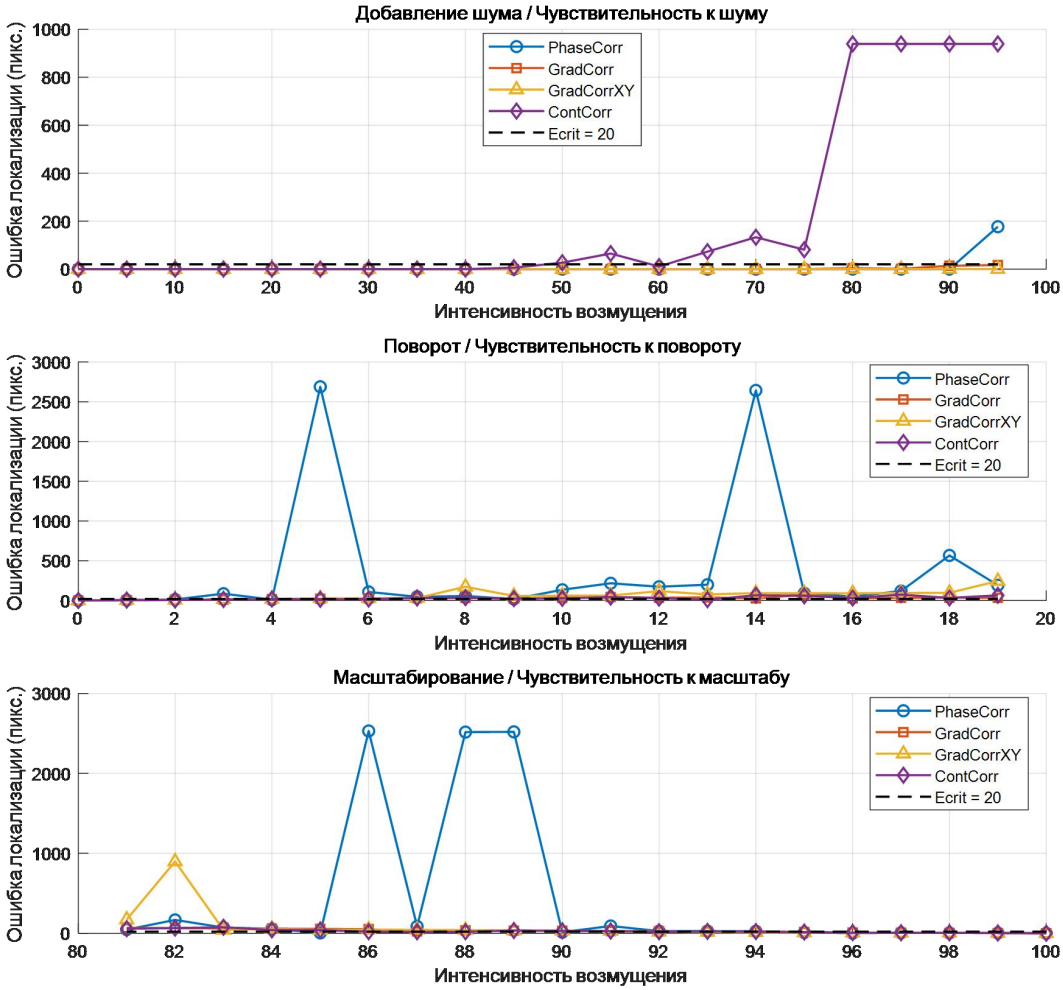
При повороте на 5°:



При масштабе 0,9:



2.6 Сравнение



Устойчивость к шуму: наилучший результат даёт градиентная корреляция, а ошибки раньше всего появляются при контурной корреляции.

Устойчивость к повороту: наилучший результат дают контурная корреляция и градиентная корреляция по модулю, а наихудший — фазовая корреляция.

Устойчивость к изменению масштаба: наилучший результат дают контурная корреляция и градиентная корреляция по модулю, а наихудший — фазовая корреляция.

3. Код

```
%% 相关性直接计算
clear;
close all;
A = imread("ConcordOrthoPhoto.png");
B = imread("westconcordorthophoto.png");
subplot(1, 2, 1), imshow(A), title('地图');
subplot(1, 2, 2), imshow(B), title('照片');
[h0, w0] = size(A);
[h1, w1] = size(B);
R = zeros(h0, w0);

timerId = tic;
for x = 1:w0-w1-1
    % fprintf('%d\n', x);
    % Read the elapsed wall-clock time in seconds and save it for later use.
    elapsedSeconds = toc(timerId);

    % Display the same duration in seconds and minutes.
    fprintf('Elapsed time: %.3f seconds (%.2f minutes)\n', ...
        elapsedSeconds, elapsedSeconds / 60);
    for y = 1:h0-h1-1
        s = 0;
        for dx = 1:w1
            for dy = 1:h1
                s = s + A(y+dy, x+dx)*B(dy, dx);
            end
        end
        R(x, y) = s;
    end
end

Rmax = max(R(:));
[y, x] = find(R==Rmax);
A(y:y+h1-1, x:x+w1-1) = B + 50;
figure('Name','已找到的位置/Найденное положение'),imshow(A);
%% 相关性频谱分析 Спектральная корреляция по яркости
clear;
close all;
A = imread("ConcordOrthoPhoto.png");
```

```
B = imread("westconcordorthophoto.png");
subplot(1, 2, 1), imshow(A), title('地图');
subplot(1, 2, 2), imshow(B), title('照片');
[h0, w0] = size(A);
[h1, w1] = size(B);

timerId = tic;
Sa = fft2(A);
Sb = fft2(B, h0, w0);
Sb1 = conj(Sb);
R = ifft2(Sa.*Sb1);

% Read the elapsed wall-clock time in seconds and save it for later use.
elapsedSeconds = toc(timerId);

% Display the same duration in seconds and minutes.
fprintf('Elapsed time: %.3f seconds (%.2f minutes)\n', ...
    elapsedSeconds, elapsedSeconds / 60);

figure, imagesc(real(R)), title("R 矩阵/Спектральная корреляция по яркости");
Rmax = max(R(:));
[y, x] = find(R==Rmax);
C = A;
C(y:y+h1-1, x:x+w1-1) = B + 50;
figure('Name','已找到的位置/Найденное положение'),imshow(C);

% 归一化光照
B = B - 20;
Sa = fft2(A);
Sb = fft2(B, h0, w0);
Sb1 = conj(Sb);
R = ifft2(Sa.*Sb1);
figure, imagesc(real(R)), title("R 矩阵/Спектральная корреляция по яркости");
Rmax = max(R(:));
[y, x] = find(R==Rmax);
D = A;
D(y:y+h1-1, x:x+w1-1) = B + 100;
figure('Name','调整光照后，已找到的位置/Найденное положение, Спектральная корреляция по
яркости'),imshow(D), title('调整光照后，已找到的位置/Найденное положение, Спектральная
корреляция по яркости');
```

```
%% 相位相关性 Фазовая корреляция
clear;
close all;
A = imread("ConcordOrthoPhoto.png");
B = imread("westconcordorthophoto.png");
subplot(1, 2, 1), imshow(A), title('地图');
subplot(1, 2, 2), imshow(B), title('照片');
[h0, w0] = size(A);
[h1, w1] = size(B);

% B=imrotate(B,5);
% B=imresize(B,0.9);
B=imnoise(B,'salt & pepper',0.97);
timerId = tic;
Sa=fft2(A);
Sb=fft2(B,h0,w0);
Sr=Sa.*conj(Sb);
Sr=Sr./abs(Sr); %Оставляем только фазу спектра
R=ifft2(Sr);

% Read the elapsed wall-clock time in seconds and save it for later use.
elapsedSeconds = toc(timerId);

% Display the same duration in seconds and minutes.
fprintf('Elapsed time: %.3f seconds (%.2f minutes)\n', ...
    elapsedSeconds, elapsedSeconds / 60);

figure, imagesc(real(R)), title("R 矩阵/Фазовая корреляция");

Rmax = max(R(:));
[y, x] = find(R==Rmax);
D = A;
D(y:y+h1-1, x:x+w1-1) = B + 100;
figure('Name','调整光照后，已找到的位置/Найденное положение'),imshow(D);

%% 梯度相关性 Градиентная корреляция
clear;
close all;
A = imread("ConcordOrthoPhoto.png");
```

```
B = imread("westconcordorthophoto.png");
subplot(1, 2, 1), imshow(A), title('地图');
subplot(1, 2, 2), imshow(B), title('照片');
[h0, w0] = size(A);
[h1, w1] = size(B);

% B=imrotate(B,5);
% B=imresize(B,0.9);
B=imnoise(B,'salt & pepper',0.97);
timerId = tic;
Ga=imgradient(A,'Sobel');
Gb=imgradient(B,'Sobel');
Sa=fft2(Ga);
Sb=fft2(Gb,h0,w0);
Sr=Sa.*conj(Sb);
R=ifft2(Sr);

% Read the elapsed wall-clock time in seconds and save it for later use.
elapsedSeconds = toc(timerId);

% Display the same duration in seconds and minutes.
fprintf('Elapsed time: %.3f seconds (%.2f minutes)\n', ...
    elapsedSeconds, elapsedSeconds / 60);

figure, imagesc(real(R)), title("R 矩阵/Градиентная корреляция");

Rmax = max(R(:));
[y, x] = find(R==Rmax);
D = A;
D(y:y+h1-1, x:x+w1-1) = B + 100;
figure('Name','调整光照后，已找到的位置/Найденное положение'),imshow(D);

%% 轮廓相关性 Контурная корреляция
clear;
close all;
A = imread("ConcordOrthoPhoto.png");
B = imread("westconcordorthophoto.png");
subplot(1, 2, 1), imshow(A), title('地图');
subplot(1, 2, 2), imshow(B), title('照片');
[h0, w0] = size(A);
```

```
[h1, w1] = size(B);

% B=imrotate(B,5);
B=imresize(B,0.9);
% B=imnoise(B,'salt & pepper',0.5);
timerId = tic;
Ea=edge(A,'Sobel');
Eb=edge(B,'Sobel');
Sa=fft2(Ea);
Sb=fft2(Eb,h0,w0);
Sr=Sa.*conj(Sb);
R=ifft2(Sr);

% Read the elapsed wall-clock time in seconds and save it for later use.
elapsedSeconds = toc(timerId);

% Display the same duration in seconds and minutes.
fprintf('Elapsed time: %.3f seconds (%.2f minutes)\n', ...
    elapsedSeconds, elapsedSeconds / 60);

figure, imagesc(real(R)), title("R 矩阵/Контурная корреляция");

Rmax = max(R(:));
[y, x] = find(R==Rmax);
D = A;
D(y:y+h1-1, x:x+w1-1) = B + 100;
figure('Name','调整光照后，已找到的位置/Найденное положение'),imshow(D);

%% 四种方法比较
clear;
close all;
A = imread("ConcordOrthoPhoto.png");
B = imread("westconcordorthophoto.png");
subplot(1, 2, 1), imshow(A), title('地图');
subplot(1, 2, 2), imshow(B), title('照片');
[h0, w0] = size(A);
[h1, w1] = size(B);
Exp = 3;
N = 90:0.25:95;
Ang=1:0.2:5;
```

```
M=95:-0.25:90;

timerId = tic;
Sa = fft2(A);
Sb = fft2(B, h0, w0);
Sb1 = conj(Sb);
Sr = Sa.*Sb1;
Sr = Sr./abs(Sr);
R = ifft2(Sr);

% Read the elapsed wall-clock time in seconds and save it for later use.
elapsedSeconds = toc(timerId);

% Display the same duration in seconds and minutes.
fprintf('Elapsed time: %.3f seconds (%.2f minutes)\n', ...
    elapsedSeconds, elapsedSeconds / 60);

Rmax = max(R(:));
[y, x] = find(R==Rmax);
xtrue = x(1); ytrue = y(1);

E = zeros(20, 1);
T = zeros(20, 1);

for pass=1:20
    if Exp==1, B1=imnoise(B,'salt & pepper',N(pass)/100);
        T(pass)=N(pass); tt='添加噪声/Чувствительность к шуму';
    elseif Exp==2, B1=imrotate(B,Ang(pass));
        T(pass)=Ang(pass); tt='添加旋转/Чувствительность к повороту';
    elseif Exp==3, B1=imresize(B,M(pass)/100);
        T(pass)=M(pass); tt='添加缩放/Чувствительность к масштабу';
    else B1=B; T(pass)=pass; tt='Не задано.';
    end

    R = PhaseCorr(A, B1);
    % R=GradCorr(A,B1);
    % R=GradCorrXY(A,B1);
    % R=ContCorr(A,B1);
    Rmax = max(R(:));
    [y, x] = find(R==Rmax);
```

```
    x = x(1); y = y(1);

    error = round(sqrt((x-xtrue)^2 + (y-ytrue)^2));
    E(pass) = error;
    fprintf('Pass=%d Err=%d\n ',pass,error);
end
Ecrit = 20*ones(size(E));
figure('Name',tt),plot(T,E,T,Ecrit),title(tt);

%% 四种方法在三种实验中的对比
clear; close all;

A = imread("ConcordOrthoPhoto.png");
B = imread("westconcordorthophoto.png");
figure('Name','原始图像');
subplot(1,2,1), imshow(A), title('地图');
subplot(1,2,2), imshow(B), title('照片');

[h0, w0] = size(A);

% ---------- 三种实验的扰动参数 ----------
N   = 0:5:100;      % 椒盐噪声百分比
Ang = 0:1:20;       % 旋转角度
M   = 100:-1:80;    % 缩放百分比
Tvals    = {N, Ang, M};
expNames = {'添加噪声 / 对噪声的敏感性', ...
            '添加旋转 / 对旋转的敏感性', ...
            '添加缩放 / 对缩放的敏感性'};
Exp = numel(expNames);
np  = 20;           % 每种实验的采样点数

% ---------- 基准真值（以相位相关为参考） ----------
timerId = tic;
Sa  = fft2(A);
Sb  = fft2(B, h0, w0);
Sr  = Sa .* conj(Sb);
Sr  = Sr ./ abs(Sr);
R   = ifft2(Sr);
elapsedSeconds = toc(timerId);
```

```
fprintf('Elapsed time: %.3f seconds (%.2f minutes)\n', ...
        elapsedSeconds, elapsedSeconds/60);
Rmax = max(R(:));
[y, x] = find(R == Rmax);
xtrue = x(1); ytrue = y(1);

% ---------- 四种方法 ----------
methods     = {@PhaseCorr, @GradCorr, @GradCorrXY, @ContCorr};
methodNames = {'PhaseCorr', 'GradCorr', 'GradCorrXY', 'ContCorr'};
styles      = {'-o', '-s', '-^', '-d'};
nM = numel(methods);

% ---------- 存储结果 E(pass, method, exp) ----------
E = zeros(np, nM, Exp);

% ---------- 三层循环 ----------
for e = 1:Exp
    for pass = 1:np
        % 构造扰动图像
        switch e
            case 1, B1 = imnoise(B, 'salt & pepper', N(pass)/100);
            case 2, B1 = imrotate(B, Ang(pass));
            case 3, B1 = imresize(B, M(pass)/100);
        end
        % 四种方法依次定位
        for m = 1:nM
            R = methods{m}(A, B1);
            Rmax = max(R(:));
            [y, x] = find(R == Rmax);
            x = x(1); y = y(1);
            E(pass, m, e) = round(sqrt((x - xtrue)^2 + (y - ytrue)^2));
        end
        % 打印本次 pass 的四方法结果
        fprintf('Exp=%d Pass=%2d | ', e, pass);
        for m = 1:nM
            fprintf('%s=%3d  ', methodNames{m}, E(pass, m, e));
        end
        fprintf('\n');
    end
end
```

```
% ---------- 绘图：3 行子图，每行一个实验 ----------
Ecrit = 20 * ones(1, np);
figure('Name','四种方法在三种实验中的对比', 'Position', [100 100 900 800]);
for e = 1:Exp
    subplot(Exp, 1, e); hold on;
    Te = Tvals{e}(1:np);                % 参数个数对齐 np
    for m = 1:nM
        plot(Te, E(:, m, e), styles{m}, 'LineWidth', 1.2, ...
             'DisplayName', methodNames{m});
    end
    plot(Te, Ecrit, 'k--', 'LineWidth', 1.2, 'DisplayName', 'Ecrit = 20');
    hold off; grid on;
    title(expNames{e});
    xlabel('扰动强度'); ylabel('定位误差 (像素)');
    legend('Location', 'best');
end
```